REJECTION-SAMPLING AND SPARSE AUTO-ENCODERS FOR INTERPRETABLE STOCK MOVEMENT PREDICTION WITH SENTIMENTS

Yik Chun Ho, Cheung Yee Lun, Lai See Ling

ABSTRACT

Stock movement prediction is a critical yet challenging task, often relying on sentiment data to capture market dynamics. However, the universal necessity of sentiment data remains underexplored. This study investigates whether sentiment is always essential for effective stock movement prediction, proposing a novel methodology that integrates rejection-sampling and sparse auto-encoders. We curate a comprehensive dataset combining sentiment from Kaggle sources with stock price data from Yahoo Finance. Using rejection-sampling, we identify and mask ”easy” time points—where pretrained models consistently predict correctly—achieving a 5% F1 Score improvement by focusing training on complex patterns. A PatchTSMixer-based ablation study evaluates the impact of sentiment and masking, while sparse auto-encoders provide interpretability by revealing when sentiment drives predictions. Our findings challenge the blanket reliance on sentiment data and offer a framework for enhanced utilization of sentiment data in time series domains.

1 INTRODUCTION

Stock movement prediction remains a formidable challenge in financial modeling, driven by the intricate dynamics of market and retail sentiments. Traditional approaches, ranging from logistic regression to deep learning, often integrate sentiment data—such as financial headlines—under the assumption that it universally enhances predictive accuracy (Xiao & Ihnaini (2023)). However, the necessity of sentiment data across all scenarios remains underexplored. We hypothesize that in cases where temporal trends are easily modeled, sentiment data may offer limited additional value. This paper seeks to address a critical question: Is sentiment data always necessary for effective stock movement prediction?

To investigate this, we propose a novel methodology combining two techniques: (1) rejection-sampling to identify and mask ”easy” time points where sentiment data may be redundant, and (2) sparse auto-encoders to interpret the model's reliance on sentiment information. Our approach not only refines prediction accuracy but also enhances the interpretability of time series classification models, offering new insights into the role of sentiment in stock prediction. Our Contributions are as below:

- **Sentiment Dataset Aggragation**: We assemble a robust dataset by merging sentiment data from multiple Kaggle sources. This curated resource supports rigorous training and evaluation of stock movement prediction models.
- **Pioneering Rejection-Sampling in Time Series**: To our knowledge, this is the first application of rejection-sampling to time series data for stock movement prediction. By masking ”easy” time points—where pretrained models consistently predict correctly—we focus training on more challenging samples, achieving a 5% improvement in F1 Score. This methodological innovation enhances model performance with sentiment data.
- **Sparse Auto-Encoders for Interpretability**: We employ sparse auto-encoders to dissect the model's dependence on sentiment data, a novel application in time series classification. This technique reveals when sentiment contributes to predictions, advancing the explainability of deep learning models in financial contexts.

2 Related Works

2.1 Time Series Foundation Models

Recent progress in pretrained time series models has focused on zero-shot forecasting, with architectures like Time-MoE leveraging mixture-of-experts designs (Shi et al. (2024)) and Amazon's Chronos (Ansari et al. (2024)) achieving state-of-the-art performance. For classification tasks, models such as TSMixer excel are commonly used to address diverse types of time series tasks (Ekambaram et al. (2023)).

2.2 Rejection-Sampling Methods

Rejection-sampling has been widely adopted in natural language processing to enhance training efficiency, as seen in Deepseek-R1 (Shao et al. (2024)) and the s1k dataset (Muennighoff et al. (2025)), which filter challenging examples to improve reasoning. We draw inspiration from these approaches, adapting rejection-sampling to time series by masking easily predictable sequences, thus sharpening the model's focus on informative data.

2.3 Sparse Auto-Encoders for Explainability

Sparse auto-encoders have gained prominence in model interpretability, notably in Google's Gemma Scope (Zhang et al. (2020)). Also, "Do I Know This Entity?" study in language models further uncovers whether the model is hallucinating via sparse autoencoder (Ferrando et al. (2024)). We extend this technique to time series, using sparse auto-encoders to probe the utility of sentiment data in stock prediction.

3 Methodology

This research investigates the question: "Is sentiment data necessary for effective stock movement prediction?" To address this, our methodology is structured into four distinct stages: Dataset Collection, Rejection-Sampling, Model Classification, and Sparse Auto-Encoder analysis. These stages systematically curate data, refine the prediction process, and interpret the model's reliance on sentiment information. The pipeline integrates sentiment and stock price data, employs rejection-sampling to focus on challenging predictions, and leverages sparse auto-encoders for interpretability. Each stage builds on the previous one, creating a robust framework for analysis.

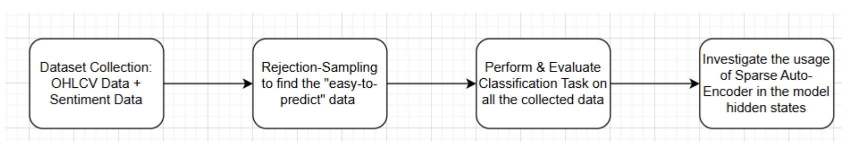

Figure 1: Pipeline Workflows

1. **Dataset Collection** - Gathering and preprocessing sentiment and stock price data.
2. **Rejection-Sampling Methods Using Pretrained Models** - Filtering out "easy" time points to enhance model focus.
3. **Stock Movement Prediction with PatchTSMixer** - Evaluating sentiment's impact via ablation studies.
4. **Sparse Auto-Encoder** - Interpreting the model's dependence on sentiment data.

3.1 Dataset Collection

We start by assembling a dataset that combines sentiment and stock price information:

- **Sentiment Dataset** - Sourced from various Kaggle datasets, this includes financial headlines, social media snippets, and other textual data reflecting market sentiment.

The data sources include:
https://huggingface.co/datasets/raeidsaqur/NIFTY
https://www.kaggle.com/datasets/notlucasp/financial-news-headlines?select=guardian_headlines.csv
https://www.kaggle.com/datasets/miguelaenlle/massive-stock-news-analysis-db-for-nlpbacktests?select=analyst_ratings_processed.csv.
The sentiment data collected are merged into one csv file with the following indicators: date, headlines, dataset name, is general (1 if the data is generally related to financial market; 0 if related to a specific stock), stock name (if the data is related to a specific stock).

- **Stock Price Data** - Open, High, Low, Close, and Volume (OHLCV) data are collected from Yahoo Finance, covering a diverse range of stocks over around 10 years period.

The sentiment data is aligned with the stock price time series to ensure temporal consistency. Preprocessing steps include cleaning the text data, normalizing stock prices with StandardScaler, and handling missing values to create a unified dataset suitable for modeling.

3.2 Rejection-Sampling Methods Using Pretrained Models

In this stage, we apply rejection-sampling to identify and mask ”easy” time points—periods where stock movement is highly predictable—allowing the model to focus on more challenging scenarios. The process is as follows:

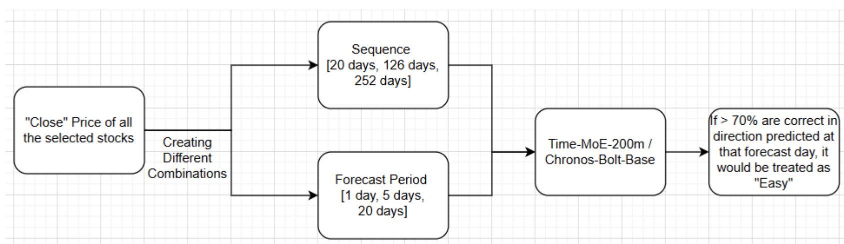

Figure 2: Rejection Sampling Process

1. **Evaluation with Pretrained Models** - Two distinct pretrained time series models are used to predict stock movement direction (up or down). Predictions are evaluated across multiple input sequence lengths (20, 126, and 252 days) and forecast horizons (5 and 20 days) - i.e., for forecast day of April 30th. If the forecast horizon is 5, it means that the data the input sequence would be around up till April 25th. If the forecast horizon is 20, then the input sequence would be filled up till April 10th.
2. **Identifying ”Easy” Time Points** - For each combination of model, sequence length, and forecast horizon, prediction accuracy is calculated. A time point is labeled ”easy” if it is correctly predicted in at least 70% of these combinations, indicating high consistency across configurations.
3. **Masking During Training** - In the training phase, the loss contribution from ”easy” time points is masked, forcing the model to prioritize learning from more complex, less predictable data. However, for the first 20% of data, there will be no masking as it requires only a few steps to learn for easy-to-predict data.

We hypothesize that this masking will be particularly beneficial when sentiment data is included, as it may reveal nuanced patterns not apparent in ”easy” periods.

3.3 Stock Movement Prediction with PatchTSMixer

We employ the PatchTSMixer architecture for stock movement classification, conducting an ablation study to assess the roles of sentiment data and rejection sampling (Ekambaram et al. (2023)). This stage includes:

- **Model Configurations** - With/Without Sentiment: Training separate models with and without sentiment data. With/Without General Sentiment: Comparing models using a general sentiment indicator versus stock-specific sentiment. With/Without Masking: Applying the rejection-sampling mask versus training on the full dataset.
- **Training Strategy** - For the initial 20% of training epochs, no masking is applied, allowing the model to learn broad patterns from the entire dataset. After this period, masking is introduced to shift focus toward challenging time points.
- **Evaluation** - Models are evaluated on a separate test dataset using accuracy and F1 score metrics. Test periods are carefully selected to avoid overlap with training data, ensuring unbiased results.

This stage quantifies the contribution of sentiment data and the effectiveness of rejection-sampling in improving prediction performance.

3.4 Sparse Auto-Encoder

To investigate the role of sentiment data in stock movement prediction, we employ sparse auto-encoders (SAEs) to analyze the internal representations of the PatchTSMixer model. Our key assumption is that sentiment information is more critical for predicting ”not easy” time points—those not consistently predicted by pretrained models—than for ”easy” time points, which are readily modeled without sentiment. By reconstructing the model’s hidden states and examining their patterns, we aim to uncover when and how sentiment influences predictions, enhancing the interpretability of time series classification.

We train SAEs to reconstruct the hidden embeddings from the PatchTSMixer model under four configurations: with/without sentiment data and with/without rejection-sampling masking. The process is as follows:

1. **Hidden State Extraction for Training**: For each sample in the training dataset, we extract the hidden states from the final layer of PatchTSMixer, capturing both general (market-wide) and stock-specific sentiment effects when applicable.
2. **SAE Architecture**: The SAE consists of an encoder-decoder pair with a sparsity constraint on the latent representation, encouraging activation of only a small subset of neurons (Ng et al. (2011), Rajamanoharan et al. (2024)). We optimize the SAE to minimize reconstruction loss while enforcing sparsity via an L1 penalty on the latent layer.
3. **Training on Training Dataset**: Separate SAEs are trained exclusively on the hidden states from the training dataset for each model configuration to learn robust representations of the model’s behavior. This ensures the SAE captures generalizable patterns before being applied to the test set.
4. **Application to Test Set**: Once trained, the SAEs are used to encode and reconstruct hidden states extracted from the test dataset, enabling analysis of unseen samples.

Using the trained SAEs, we analyze the test dataset to discern patterns in model behavior across different prediction scenarios. The analysis focuses on four categories of test samples, defined by prediction difficulty and outcome:

- **Easy + Correct**: Easy time points (via rejection-sampling) and correctly predicted.
- **Easy + Incorrect**: Easy time points incorrectly predicted.
- **Hard + Correct**: Non-easy time points correctly predicted.
- **Hard + Incorrect**: Non-easy time points incorrectly predicted.

For each category, we would perform the **Supervised Classification** using the SAE embeddings as features, we train a Random Forest model to classify test samples into the four categories (Easy + Correct, etc.) (Breiman (2001)). Classification performance (e.g., accuracy, F1 score) indicates how well SAE embeddings could identify the correct prediction samples. If there are a high accuracy of predicting hard and correct samples based on the SAE embeddings, we could make a more confident prediction during inference via SAE.

4 EXPERIMENTS AND RESULTS

4.1 EXPERIMENTS SETUP

We conduct experiments to classify stock price movements into three categories: rise (more than +0.2%), neutral (otherwise), or drop (lower than -0.2%), over a 20-day forecast horizon using a PatchTSMixer model. The dataset, derived from multiple Kaggle data sources and yahoo-finance, includes daily OHLCV data, stock-specific sentiment embeddings (768-dimensional), general sentiment embeddings, and an easy indicator for each forecast day. Split temporally at January 1, 2017, the training set covers pre-2017 data, and the test set includes 2017 onward. Sequences of 252 days are constructed, with OHLCV features standardized via Standard Scaler and sentiment embeddings are computed day-by-day for each of the forecasting day using FinBERT, it is cached in separate .npz files for train and test sets to optimize loading.

Our dataset ranges from '2010-01-06' to '2020-07-19' using 50 stocks selected from the S and P 500. For stocks without data within this range, we would just simply not considering the stock for that interval. The PatchTSMixer model, configured with a context length of 252, patch length of 12, 32-dimensional embeddings, and 8 layers, processes 5–15 input channels based on ablation settings: 15 (OHLCV + stock-specific + general sentiment) when with sentiment=True and use general=True, 10 (OHLCV + stock-specific) when use general=False, or 5 (OHLCV only) when with sentiment=False. A shared nn.Linear(768, 5) projects sentiment embeddings when used. Training uses Adam (learning rate 1e-4) for 20 epochs with a batch size of 32 and cross-entropy loss. A masking strategy includes all samples for the first 20% of steps, then masks the loss for easy samples (with mask=True) to focus on challenging ones.

Evaluation reports accuracy and weighted F1 score on the test set. An ablation study assesses three factors: stock-specific sentiment (with sentiment), general sentiment (with general), and masking (mask). Six configurations are tested: combinations of with sentiment=True with use general=True/False and with mask=True/False, plus two with sentiment=False and with mask=True/False. Models and projections are saved for analysis, with results summarized to compare the impact of sentiment and masking strategies.

4.2 CLASSIFICATION RESULTS

Experiment	Accuracy	F1 Score
With_Sentiment_With_General_Mask	0.4828	0.4736
With_Sentiment_With_General_NoMask	0.4747	0.4639
With_Sentiment_No_General_Mask	0.5242	0.4416
With_Sentiment_No_General_NoMask	0.5533	0.4153
Without_Sentiment_Mask	0.5494	0.4068
Without_Sentiment_NoMask	0.5566	0.4119

Table 1: Ablation Study Results for Stock Price Movement Classification

This table includes three columns: Experiment, Accuracy, and F1 Score, with each row corresponding to a specific experiment configuration. The ablation study results reveal several key insights about the impact of sentiment embeddings and masking strategies on stock price movement classification:

- **Sentiment Boosts Performance:** Experiments incorporating sentiment embeddings (`With_Sentiment_*`) generally outperform those without (`Without_Sentiment_*`), especially in F1 score. For instance, `With_Sentiment_With_General_Mask` achieves an F1 score of 0.4736, compared to 0.4068 for `Without_Sentiment_Mask`, highlighting the predictive power of sentiment information.
- **General Sentiment Enhances Results:** Including both stock-specific and general sentiment (`With_Sentiment_With_General_*`) yields higher F1 scores than using only stock-specific sentiment (`With_Sentiment_No_General_*`). For example, `With_Sentiment_With_General_Mask` (F1: 0.4736) outperforms

With_Sentiment_No_General_Mask (F1: 0.4416), suggesting that general sentiment captures valuable market-wide trends.

- **Masking Trade-offs:** When sentiment is included, masking improves F1 scores but slightly lowers accuracy. Compare With_Sentiment_With_General__Mask (Accuracy: 0.4828, F1: 0.4736) to With_Sentiment_With_General__NoMask (Accuracy: 0.4747, F1: 0.4639). Without sentiment, masking slightly reduces both metrics, as seen in Without_Sentiment_Mask (Accuracy: 0.5494, F1: 0.4068) versus Without_Sentiment_NoMask (Accuracy: 0.5566, F1: 0.4119), which validates our hypothesis that we need to mask the easy data in the case with sentiments.

4.3 Sparse Auto-encoder Results

To evaluate the role of sentiment data in predicting stock price movements, we conducted experiments using a Sparse Auto-Encoder (SAE) to interpret the internal representations of a PatchTSMixer model. The SAE, configured as a TopKSAE with a hidden dimension of 8000 (d-sae=8000) and top-k activation of 160 (k=160), was trained on hidden states extracted from the last layer of the PatchTSMixer model. These hidden states were derived from two distinct conditions: one incorporating both stock-specific and general sentiment data (withsentiment-general-withmask), and another using only stock-specific sentiment (withsentiment-withmask). The dataset, split at January 1, 2017, into training (pre-2017) and test (post-2017) sets, was processed with a sequence length of 252 days and a forecast horizon of 20 days. For each condition, we classified samples into four categories—Easy + Correct, Easy + Incorrect, Hard + Correct, and Hard + Incorrect—based on prediction difficulty and accuracy in the testing dataset. These SAE-generated embeddings were then subjected to supervised classification using a Random Forest model to assess their ability to distinguish between these categories.

The following table summarizes the Random Forest classification performance for SAE embeddings under two conditions: with general sentiment (withsentiment-general-withmask) and without general sentiment (withsentiment-withmask). Metrics include precision, recall, F1-score, and support for each category, along with overall accuracy, macro average, and weighted average.

Table 2: Random Forest Classification Performance for SAE Embeddings

Condition	**Category**	**Precision**	**Recall**	**F1-Score**	**Support**
With General Sentiment	Easy + Correct	0.00	0.00	0.00	1719
	Easy + Incorrect	0.00	0.00	0.00	1517
	Hard + Correct	0.42	1.00	0.60	9629
	Hard + Incorrect	0.41	0.00	0.01	9836
	Accuracy		0.42		22701
	Macro Avg	0.21	0.25	0.15	22701
	Weighted Avg	0.36	0.42	0.26	22701
Without General Sentiment	Easy + Correct	0.00	0.00	0.00	1582
	Easy + Incorrect	0.00	0.00	0.00	1654
	Hard + Correct	0.47	0.97	0.63	10609
	Hard + Incorrect	0.40	0.03	0.06	8856
	Accuracy		0.47		22701
	Macro Avg	0.22	0.25	0.17	22701
	Weighted Avg	0.38	0.47	0.32	22701

The Random Forest classification results indicate that the SAE embeddings provide limited separability across the four categories, with notable differences between the two sentiment conditions. In the model with general sentiment, the ”Hard + Correct” class achieved a precision of 0.42 and recall of 1.00 (F1-score: 0.60), while ”Easy” classes showed zero precision and recall, suggesting poor differentiation. Without general sentiment, ”Hard + Correct” improved slightly (precision: 0.47, recall: 0.97, F1-score: 0.63), and ”Hard + Incorrect” had a precision of 0.40 but a low recall of 0.03, with ”Easy” classes again at zero. Overall macro F1-scores were low (0.15 with general sentiment, 0.17 without), highlighting that the SAE embeddings capture some features relevant to correct predictions in harder cases, particularly without general sentiment. However, the inability to distinguish ”Easy” categories and inconsistent performance on ”Incorrect” predictions suggest that while sentiment data aids interpretability in complex scenarios, further refinement of the SAE or model architecture is needed for robust classification across all cases.

5 Conclusion

This study addresses the pivotal question of whether sentiment data is indispensable for stock movement prediction, demonstrating that its utility depends on the predictability of temporal trends. By introducing rejection-sampling to mask easily predictable time points, we enhance model performance by 5%, focusing learning on challenging market scenarios. The integration of sparse autoencoders further illuminates the model's dependence on sentiment, providing interpretable insights into when and why sentiment matters. Our ablation study with PatchTSMixer confirms that selective masking and sentiment inclusion yield superior accuracy and F1 scores, particularly for complex time series like stock prices.

The potential applications of this work are far-reaching. For financial practitioners, our approach enables more precise trading strategies by identifying when sentiment data adds predictive value. Beyond finance, the methodology can be adapted to other time series tasks, such as energy demand forecasting or healthcare trend analysis, where distinguishing easy and complex patterns is critical. Future research could explore diverse rejection-sampling strategies, such as adaptive thresholds based on market volatility or multi-model consensus for identifying ”easy” points, to further refine training efficiency. Similarly, advancements in sparse auto-encoder design—such as incorporating temporal dependencies or hierarchical sparsity—could deepen interpretability, enabling finer-grained analysis of model behavior. By bridging prediction and explanation, our framework paves the way for more robust and transparent time series modeling across domains.

Acknowledgments

We acknowledge the use of ChatGPT and Grok to plan my essay, generate some ideas for the content, polish the content and latex format for the outputs, and implement the codes with our supervision. This report was generated by an ChatGPT and Grok with our help.

References

Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*, 2024.

Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001.

Vijay Ekambaram, Arindam Jati, Nam Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. Tsmixer: Lightweight mlp-mixer model for multivariate time series forecasting. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*, pp. 459–469, 2023.

Javier Ferrando, Oscar Obeso, Senthooran Rajamanoharan, and Neel Nanda. Do i know this entity? knowledge awareness and hallucinations in language models. *arXiv preprint arXiv:2411.14257*, 2024.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

Andrew Ng et al. Sparse autoencoder. *CS294A Lecture notes*, 72(2011):1–19, 2011.

Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. Time-moe: Billion-scale time series foundation models with mixture of experts. *arXiv preprint arXiv:2409.16040*, 2024.

Qianyi Xiao and Baha Ihnaini. Stock trend prediction using sentiment analysis. *PeerJ Computer Science*, 9:e1293, 2023.

Chi Zhang, Shuang Liang, Fei Lyu, and Libing Fang. Stock-index tracking optimization using auto-encoders. *Frontiers in Physics*, 8:388, 2020.